AINMM：存量生产级工程向 AI Native 演进的五级成熟度模型

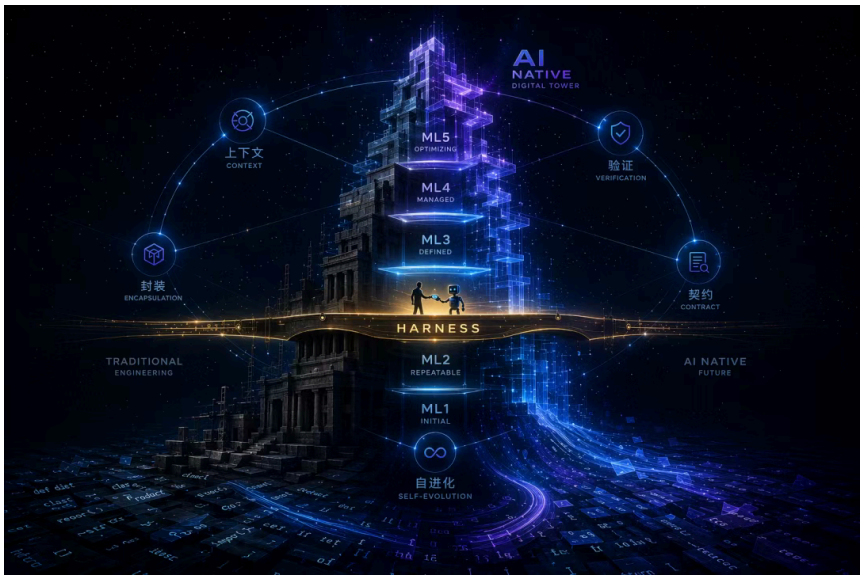
原创 供给技术 大淘宝技术 2026年7月14日 23:46 浙江 11人

本文提出 AI Native 能力成熟度模型(AINMM)，为软件工程组织提供AI原生研发能力的评估与提升框架。AINMM 借鉴 CMMI 思想，定义了五个成熟度等级(ML1-ML5)：从 ML1 “AI 认识项目”到 ML5 “组织级自进化”，围绕五大过程域(上下文工程、能力封装、验证回路、协作契约、自进化)构建。模型针对存量工程转型痛点，提供可测量、可比较、可指导的渐进式路径，避免“big bang”式重构。配套 AI Native Evolution Kit 工具支持自动化评估与演进。通过挽单系统实践验证，该框架能准确定位团队 AI 能力短板，指导从“AI 辅助”向“全链路 AI 驱动”的系统性转型，助力实现十倍级效率跃迁。AINMM 不是银弹，而是随实践持续进化的工程指南。

01 前言

> 声明：1、本文内容由作者和AI协作完成，大致分工：我负责整体框架构思（从背景→思想→定义→路径→评估→案例）、案例实操（踩坑→改造→洞察）、最终修改；AI负责材料收集、CMMI模仿、YY数据制表、制图、文字润色；2、本文（尝试）为完整阐述所提出的五层模型，涉及较多概念，行文略显详尽；如需快速把握核心价值，建议直接阅读第七章（模型的作用与价值）、第八章（具体案例）。

副标题：AI Native Maturity Model (AINMM) — 定义文档 v1.0



本文的写作契机，源于公司在年度战略会上，管理层对全集团提出的 **AI Native** 战略要求。讲话中，将 AI Native 定义为组织形态升级的核心标准：任何应用、任何任务，都必须拆解到全链路 AI 驱动的流程中；Agent 与 Agent 之间通过标准化的协议和上下文进行交互，而非靠人工开会串联；最终目标是实现10倍级的生产效率跃迁，让产品具备“每天一个版本”的迭代能力。这一要求不仅指向从零开始的 Agentic 原生项目，更关键的是——**如何让我们大量随业务发展沉淀下来的存量生产级工程，逐步演进为 AI Native 工程**。

在实践层面，我们团队一方面在 Agentic 原生项目中已经积累了较为成熟的 AI Native 经验；另一方面，也切身感受到存量工程转型的复杂性与特殊性。不同于0-1就是Agentic的项目，这些工程往往具有以下特征：**工程面向人开发和维护**，缺乏面向 AI 的上下文环境与执行约束；**生产级稳定性要求高**，任何改动都需要经过严格的回归与验证；**端分离明显、系统组成复杂**，不少项目横跨多个组织、涉及多端（前端、后端、移动端等）交付；**需求长期处于持续迭代状态**，不能承受"停下来重构"的成本。正因如此，存量工程的 AI Native 转型不能完全照搬新项目的AI Native方式，它需要一套**可测量、可比较、可指导**的渐进式框架——告诉团队"现在在哪""缺口是什么""下一步该做什么"。

本文提出的 **AI Native 能力成熟度模型（AINMM：AI Native Maturity Model）**，正是基于上述背景，结合 **Vibe Coding**、**Spec 驱动开发**以及 **Harness Engineering**实践和思考后，所形成的一套评估与提升框架。模型中的每一个等级定义、每一条实践路径、每一组度量指标，源于所维护的两个生产级项目在向 AI Native 演进过程中的真实踩坑与阶段性沉淀。

最后需要说明的是：**没有一种范式是银弹**。AINMM 提供的是一个通用参考框架，而非必须逐字照搬的标准答案。不同团队的业务特点、技术债规模、人员情况各不相同，在实际落地时，需要结合改造 ROI 和团队现状进行裁剪与适配。欢迎大家反馈与实践补充，一起推进存量项目向AI Native转型，我也会持续实践并结合踩坑经验持续打磨这个通用框架，让这个框架在真实的工程土壤中不断进化。

02 概述

什么是 AI Native 能力成熟度模型

AI Native 能力成熟度模型（AI Native Maturity Model，简称 AINMM）是一套用于评估和指导**软件工程组织在 AI 原生研发范式下成熟程度**的结构化框架。它借鉴了 CMMI（Capability Maturity Model Integration，能力成熟度模型集成）的核心思想，针对"AI Agent 参与软件研发全生命周期"这一新工程范式，定义了五个成熟度等级（ML1—ML5），以及每个等级需要达成的目标、关键过程域、度量标准和提升路径。模型假设组织已引入 AI 参与研发过程，评估的是"AI 参与的深度和工程化程度"，而非"是否引入 AI"。

为什么需要 AINMM

当前行业面临一个结构性矛盾：**AI 的代码产出能力在快速增长，但团队的端到端交付效率并未同步提升**。随着AI的发展越来越快，尽管部分统计数据存在滞后性，但为了体感更量化，我和AI整理了下面几组可供参考的数据进行说明。Sonar 2025 年度报告显示全球新增代码中 42% 由 AI 辅助生成[1]，GitClear 2025 年研究指出代码搅动率（代码被频繁修改、删除或重写的程度）近乎翻倍、重复代码增长近四倍，开发者 Review 时间首次超过编写时间[2]。Stack Overflow 2025 开发者调查进一步显示，84% 的开发人员已在使用或计划使用 AI 编码工具，但仅有 29% 表示信任 AI 产出，信任度较上年下滑 11 个百分点[3]。OpenAI 在 2026 年初正式将这个缺口定义为 **Harness Gap**[4]。

问题在于：大多数组织对自身的 AI 协作能力缺乏客观、可量化的评估手段。团队的状态描述往往是模糊的——"我们在用 AI"、"效率提升了一些"、"AI 替代了 XX% 的工作"——这些说法既无法横向比较，也无法指导改进方向。

AINMM 的目标就是把这些模糊感知变成**可测量**、**可比较**、**可指导**的工程框架，让组织能够：

1. **准确定位**当前 AI Native 能力所处的成熟度等级
2. **明确差距**识别当前等级与目标等级之间的关键能力短板
3. **规划路径**获得从当前等级提升到下一等级的具体改进路线
4. **度量进步**用客观指标跟踪改进效果
5. **横向对标**在组织内部不同团队之间建立统一的评估语言



适用范围与局限性

适用范围：AINMM 主要面向具备一定规模和复杂度的软件研发组织——典型画像是拥有多人以上研发团队、维护至少一个生产级系统（数百个源文件以上）、使用 CI/CD 流程的团队。技术栈不限（Java、Python、Node、Go 等均适用），部署环境不限（云原生、传统 IDC、混合部署均可）。

已知局限：第一，AINMM 当前版本基于作者实践基础上，结合AI对阿里组织内部和开源社区的提炼，在跨行业（航空航天、医疗设备、嵌入式系统）等领域的适用性尚未知。第二，模型假设已具备基本的版本控制和 CI/CD 基础设施——对于尚未建立这些基础的组织，需先补齐传统工程能力再评估 AI Native 成熟度（即没有设定ML0，保留5级与CMMI形成参照）。第三，五维度评分标准在评分颗粒度上仍有提升空间（详见第六章评估方法），不同评估者之间可能存在一定的主观偏差，这部分可以由具体团队、具体项目在通用框架基础上进行差异化定制。第四，框架评估脚本按照通用习惯和个人喜好进行命名，因此可能存在脚本评分偏差（例如工程真实存在AI上下文文件，但非AGENTS.md格式则同样不计分）。

目标读者

本文面向以下角色，不同读者可按需选择阅读路径：

技术 Lead / 架构师——本文的核心读者。关心"怎么系统性地把 AI 融入团队的日常研发流程"，建议通读全文，重点关注第三章（五级定义）、第五章（提升路径）和第六章（评估方法）。拿到文章配套的 Evolution Kit 工具后，可以直接用 `audit.sh` 对你的项目做第一次评估。

具体开发者——关心"AI 协作到底怎么干、有没有可执行的东西"，建议先看第三章的 ML1-ML2 定义了解起步标准，然后跳到第五章 5.1-5.2 看具体操作步骤。配套代码库中的 `init-workspace.sh` 和 `evolve.sh` 可以帮你快速搭建 AI Native 基础设施。

研发总监（AI YY）——关心"怎么量化团队的 AI 能力、怎么横向比较、投入产出比"，建议重点看第一章（为什么需要 AINMM）、第四章（等级总览）和第七章（模型价值）。成熟度等级和雷达图评分可以直接用于组织级的对标和汇报。

AI 基础设施开发者——关心"怎么让自己的 AI 工具/基础设施更好地融入企业研发流程"，建议看第二章基本思想，以及第三章中各等级的过程域要求——你的工具/基础设施应该帮助用

户向更高等级进阶。

配套工具：AI Native Evolution Kit

本文定义了 AINMM 的理论模型，而 **AI Native Evolution Kit** 是该模型的工程实现——一套开源的命令行工具框架，支持从评估到生成到持续进化的全链路操作。

```
# 克隆工具框架
git clone git@gitlab.alibaba-inc.com:top/ai-native-evolution-kit.g
# 审计你的项目（输出五维度评分 + ML 等级）
./audit.sh /path/to/your-project --detail
# 一键初始化 AI Native 工作空间
./init-workspace.sh
# 深度分析 + 生成 Harness 五层基础设施
./evolve.sh /path/to/your-project --output /path/to/workspace
```

工具	对应 AINMM 环节	功能
`audit.sh`	第六章 AINA-C 自动评估	D1-D5 五维度打分，输出 ML1-ML5 等级和改进建议
`evolve.sh`	第五章 提升路径	扫描项目现状 → 分析差距 → 生成 Harness Layer 1-5 基础设施
`init-workspace.sh`	第五章 5.1-5.2	一键创建虚拟 Monorepo 工作空间（推荐入口）
`generators/generate-skill.sh`	第三章 ML2 能力封装	快速创建新 Skill 的目录和模板
`templates/`	第三章 全部等级	Harness Layer 1-5 的标准化模板（.tmpl）

文章中涉及的所有概念——五大过程域（PA1-PA5）、五个评估维度（D1-D5）、成熟度等级（ML1-ML5）、8 阶段 SOP——目前均与代码库的命名和逻辑保持一致，统一维护。

AINMM 与 CMMI 的关系

AINMM 是 借鉴CMMI 思想在 AI Native 研发领域的探索应用[5]，为了实现这一目标，尝试定义二者的关系，如下：

维度	CMMI	AINMM
评估对象	软件开发与工程过程的成熟度	AI Agent 参与研发全生命周期的成熟度
核心问题	如何让软件过程可预测、可控制	如何让非确定性 AI 项目工程在确定性生产管道中可靠性运行
等级数量	5 级（ML1—ML5）	5 级（ML1—ML5），与 CMMI 保持一致
关键过程域	22 个传统软件过程域	5 大过程域（上下文工程、能力封装、验证回路、协作契约、自进化）
表示方式	阶段式 + 连续式	阶段式（整体等级）+ 雷达图式（五维度独立评分）
评估方法	SCAMPI	AINA（AI Native Assessment，五维度 0-100 分评估）

关键继承：CMMI 的"逐级递进、每级是下一级基础"原则完整保留——ML1 是 ML2 的基础，ML2 是 ML3 的基础，不存在跳级的捷径。AINMM 与 CMMI 保持相同的五级结构，模型假设组织已引入 AI 参与研发，ML1 作为起始等级评估 AI 协作的初始工程化水平。

03 基本思想

设计哲学

AINMM 的核心设计哲学可以用三句话总结：

第一，Harness 比 Agent 更重要。 模型在快速迭代发展，今天的限制明天可能消除，今天的提示词可能在明天模型升级后更简单。但 Harness是稳定的控制面，不依赖特定模型。AINMM 评估的不是"用了多先进的 AI 模型"，而是"为 AI 搭建了多完善的工作环境"。

第二，信任是可以工程化的。 不需要全信 AI 或全不信 AI。通过上下文工程、能力集成封装、验证回路、协作契约和自进化机制，信任可以被分级、被验证、被逐步释放。AINMM 的每个等级本质上都是在回答"这个组织在多大程度上能安心地把工作交给 AI"。

第三，成熟度是渐进式的螺旋上升，尤其在存量生产应用中，不是 big bang 式的跃进。 每一级独立有价值，可持续叠加优化。我们不需要"等条件成熟"再开始，从 AGENTS.md 或者一个wiki开始就是一个有价值的起点。



两种表示法

仿照 CMMI 的双表示法设计，AINMM 提供两种互补的评估视角：

阶段式表示法（Staged Representation）——组织整体成熟度等级。将组织的 AI Native 能力整体评定为 ML1—ML5 中的某一级，适用于组织级别的对标、汇报和战略规划。等级

判定规则：**取最低满足的等级**——即所有关键过程域都达标的最高等级。

雷达图式表示法（Radar Representation）——五维度独立评分。对五大过程域（上下文工程 D1、能力封装 D2、验证回路 D3、协作契约 D4、自进化 D5）分别独立评分（每维度 0-20 分，总分 0-100 分），适用于识别具体短板、制定针对性改进方案。一个项目可能 D1 得 16 分但 D3 只有 4 分——雷达图能精确呈现这种不均衡。

两种表示法的关系：**阶段式等级由雷达图评分推导**。当五个维度的得分都满足某一等级的阈值要求时，该等级才成立。

五大过程域

AINMM 将 AI Native 研发能力分解为五大过程域（Process Area，PA），每个过程域对应 Harness 五层架构中的一层：

过程域编号	过程域名称	对应 Harness 层	核心问题
PA1	上下文工程（Context Engineering）	Layer 1	AI 能否"认识"项目?
PA2	能力封装（Capability Encapsulation）	Layer 2	AI 有没有"可控的手脚"?
PA3	验证回路（Verification Loop）	Layer 3	AI 的产出"可信任"吗?
PA4	协作契约（Collaboration Contract）	Layer 4	人和 AI 边界是什么？边界清晰吗?
PA5	自进化（Self-Evolution）	Layer 5	系统越用越好吗？自进化是可控的吗?

每个过程域包含：**特定目标**（Specific Goals，SG）、**特定实践**（Specific Practices，SP）、**通用目标**（Generic Goals，GG）和**度量指标**（Metrics）。



ML1：已感知级（Aware）

一句话定义：已认识到 AI 开发的价值，开始建立基础上下文，同样的领域内容不用重复告诉AI，AI 能"认识"项目但尚无可控的执行能力。

典型特征：

- 项目具备 AGENTS.md 或等效的结构化上下文文档，AI Agent 能理解项目的基本结构
- 上下文文档遵循"地图而非手册"原则——控制在 200 行内，索引指路而非面面俱到
- 关键编码规范、架构约定、禁止事项已落入仓库文档，不再纯靠口口相传
- AI 的使用仍然是"辅助模式"——人写需求、AI 补全或生成片段、人全程审查
- 没有形成标准化的 Skill，AI 的行为不可预测和复制

五维度基准分：D1≥8, D2<8, D3<4, D4<4, D5<4（总分约 8-16 分）

关键过程域要求：

过程域	特定目标	特定实践
PA1 上下文工程	SG1: 仓库即真相源（Repository as Single Source of Truth）	SP1.1: 创建 AGENTS.md，包含项目概览、技术栈、目录结构、编码规范、禁止事项
		SP1.2: 建立 `docs/` 目录，结构化存放设计文档、架构说明、中间件参考
		SP1.3: 信息分级——"AI 不知道就会写错"的放 AGENTS.md，"只是写不够好"的放 docs 链接
	SG2: 新人（人类或 AI）能通过读文档上手	SP2.1: AI Agent 能正确回答"项目做什么、用什么技术栈、哪些事不能做"
PA2-PA5	无强制要求	可有个别 Skill 或工具使用，但不作为等级评估条件

通用目标：GG1——AI 能"认识"项目，不需要人在对话框中反复补充背景，节省人的时间和交接成本。

度量指标：

指标	定义	ML1 基准值
上下文覆盖率	AGENTS.md + docs/ 覆盖的关键领域知识占比	≥60%
AI 项目认知准确率	AI 回答项目基础问题的正确率	≥80%
上下文文档新鲜度	文档最近一次更新距今天数	≤30 天

判定标准：PA1 的所有特定目标和特定实践已达成，AI 能正确理解项目全貌。

ML2：已管理级（Managed）

一句话定义：AI 的执行能力被结构化封装为可复用的 Skill，产出通过自动化门禁验证，AI 从"辅助工具"进化为"可管理的执行者"。

典型特征：

- 高频操作被封装为标准化 Skill（SKILL.md），定义了触发条件、执行步骤、验证标准和已知陷阱
- AI Agent 执行时调用对应 Skill，行为可预测、可复制
- 建立了基础的自动化验证门禁（至少覆盖编译检查和架构约束验证）
- AI 产出不再完全依赖人工 Review，门禁系统承担了一部分质量保障
- 经验开始沉淀到 MEMORY.md 和 known-bad-patterns，但尚未系统化
- AI 能"触达"生产系统——如协议桥、API 网关等基础设施就位

五维度基准分：D1≥12, D2≥8, D3≥8, D4<8, D5≥4（总分约 32-44 分）

关键过程域要求：

过程域	特定目标	特定实践
PA1 上下文工程	ML1 全部 + SG3: 按需加载	SP3.1: 详细文档按需加载，不一次性灌入全部上下文
		SP3.2: 领域模型文档（domain-model.md）就位
PA2 能力封装	SG1: 高频操作 Skill 化	SP1.1: 至少 3（这个指标是随便拍的，根据项目可自行调整） 个核心 Skill 已定义（含触发条件、步骤、验证标准、Gotchas）
		SP1.2: Skill 遵循五种设计模式之一（Tool Wrapper / Generator / Reviewer / Inversion / Pipeline）
	SG2: AI 能触达生产系统	SP2.1: AI Agent 能执行端到端验证（如调用接口、运行测试）
		SP2.2: 协议桥或 API 网关等基础设施就位（如需要）
PA3 验证回路	SG1: 基础门禁就位	SP1.1: 编译检查（Gate 1）自动化
		SP1.2: 架构约束验证（Gate 2）自动化
		SP1.3: 门禁报错信息包含"什么规则违反了 + 为什么是问题 + 怎么修"
	SG2: 存量项目基线门禁	SP2.1: 对已有问题建立基线，只报新增 issue

通用目标：GG2——AI 的核心操作可预测、可复制、可验证。

度量指标：

指标	定义	ML2 基准值
Skill 覆盖率	高频操作中已被 Skill 封装的占比	≥50%
Gate 通过率	AI 首次提交通过自动化门禁的比例	≥60%
AI 自修复成功率	Gate 失败后 AI 自行修复通过的比例	≥40%
MEMORY.md 条目数	经验沉淀的条目数量	≥20 条

判定标准：PA1 和 PA2 的所有特定目标已达成，PA3 的基础门禁已就位。

ML3：已定义级（Defined）

一句话定义：人机协作边界通过契约式设计明确定义，AI 在约定的范围内可自主执行，组织级标准流程形成。

典型特征：

- 协作契约通过 Design by Contract 原则定义——前置断言、后置断言、不变式
- 置信度路由机制运行——高置信度任务（>90%）自动推进，中置信度（60-90%）标记审查，低置信度（30-60%）暂停等人，极低置信度（<30%）拒绝执行

- 8 阶段协作 SOP 标准化运行（目标收敛 → 状态恢复 → 上下文装配 → 任务分块 → 链路设计 → 执行前校准 → 外部验证 → 回写）
- 验证门禁覆盖全链路（编译 → 架构 → 单元测试 → 集成测试 → 端到端验证）
- 知识资产化开始系统化——执行过程中的知识自动沉淀（不依赖人工记忆）
- AI 协作流程在组织层面标准化——不同团队遵循相同的基本框架

五维度基准分： D1≥16, D2≥12, D3≥12, D4≥10, D5<8（总分约 50-64 分）

关键过程域要求：

过程域	特定目标	特定实践
PA1-PA3	ML2 全部 + 增强	PA3 增强: 门禁覆盖 Gate 1-4 全链路，AI 自修复成功率 ≥60%
PA4 协作契约	SG1: 契约式协作	SP1.1: 前置断言——AI 执行前必须复述目标，断言不通过拒绝执行
		SP1.2: 后置断言——完成后必须通过外部证据（测试/日志/回包）验证
		SP1.3: 不变式——外部 Spec 不容篡改，跨 session 状态由文档保持
	SG2: 分级信任机制	SP2.1: 置信度路由机制运行（四级分流）
		SP2.2: 角色职责矩阵定义（哪些环节 AI 自主、哪些人介入）
	SG3: 标准化协作流程	SP3.1: 8 阶段 SOP 在团队中统一执行
		SP3.2: 偏航检测信号定义并运行（绕过阶段目标、跳过中间产物、主观替代客观、混淆阶段/全局完成）

通用目标： GG3——人机协作边界清晰，AI 在约定范围内可自主运行，异常时按协议升级。

度量指标：

指标	定义	ML3 基准值
Gate 全链路通过率	AI 提交通过 Gate 1-4 全部门禁的比例	≥70%
AI 自主完成率	AI 无需人工干预即可完成的任务占比	≥40%
置信度路由准确率	AI 自评置信度与实际通过率的一致性	≥70%
人工 Review 修改率	Review 后需修改的 AI 产出占比（越低越好）	≤30%
偏航检测触发率	偏航信号被正确触发的比例	≥80%
知识资产化率	Gate 失败/Review 修改后经验被沉淀的比例	≥60%

判定标准： PA1-PA4 的所有特定目标已达成，PA5 有初步实践但不作为硬性要求。

ML4：已量化级（Quantitatively Managed）

一句话定义：AI 协作过程实现数据驱动的量化管理，自进化机制开始运行，关键指标可预测。

典型特征：

- 所有关键指标（Gate 通过率、Review 修改率、AI 自主完成率、举手频率等）持续采集和监控
- 自进化机制从被动积累（Level 1）进化到主动优化（Level 2-3）——定期 Skill 审查 + 离线优化
- 自进化与主运行时严格解耦——运行时产生轨迹，离线进化器分析优化，人工确认后写回
- 量化目标驱动优化——不再是"感觉变好了"，而是"Gate 通过率从 70% 提升到 85%"
- 对抗式审查机制运行——AI 审 AI（Critic Agent），加上橡皮图章检测（大 diff + 零 issue = 可能走过场）
- Evolution Kit 等标准化工具在组织内推广使用

五维度基准分：D1≥18, D2≥16, D3≥16, D4≥12, D5≥8（总分约 70-85 分）

关键过程域要求：

过程域	特定目标	特定实践
PA1-PA4	ML3 全部 + 增强	PA3 增强: 对抗式审查（Critic Agent）运行；PA4 增强: 置信度阈值基于历史数据持续校准
PA5 自进化	SG1: 运行时知识资产化（Level 1）	SP1.1: Gate 失败 → 自修复 → 修复模式自动记入 MEMORY.md
		SP1.2: Review 修改 → 修改模式自动记入 known-bad-patterns
		SP1.3: 新发现的边界条件追加到测试场景库
	SG2: 结构化审查（Level 2）	SP2.1: 定期（每 Sprint）执行 Skill 质量审查（7 维度评估）
		SP2.2: 审查结果形成优化提案，经人工确认后应用
	SG3: 数据驱动优化（Level 3）	SP3.1: 自进化优化循环（生成方案 → 评估效果 → 提出改进 → 应用验证）跑通
		SP3.2: 进化与运行时解耦——离线分析轨迹、生成方案、人工确认后写回
		SP3.3: 进化的元约束就位（渐进性、可逆性、透明性、灰度性、人类否决权）

通用目标：GG4——AI 协作过程可量化、可预测，进化受控且有数据支撑。

度量指标：

指标	定义	ML4 基准值
ML3 全部指标	持续满足	Gate 通过率 ≥85%，AI 自主完成率 ≥60%
Skill 审查覆盖率	已接受过结构化审查的 Skill 占比	≥80%
自进化优化循环完成次数	每季度完成的自进化优化循环数	≥2 次
进化可逆率	进化操作可被安全回滚的比例	100%
Critic 审查拦截率	Critic Agent 在人工 Review 前发现问题的比例	≥50%
指标趋势方向	关键指标是否持续向好	连续 3 个 Sprint 稳步提升

判定标准：PA1-PA5 的所有特定目标已达成，关键指标实现持续量化跟踪。

ML5：持续优化级（Optimizing）

一句话定义：AI Native 能力实现组织级持续优化，进化成果可复制到新项目，组织形成自我演进的 AI Native 生态系统。

典型特征：

- 低风险进化自动生效（如 Gotchas 追加、MEMORY.md 精炼），高风险进化经人工确认
- 个体项目的进化成果可复制——例如挽单优化出的 patterns、Skill、阈值、最佳实践能复用给下一个项目
- Evolution Kit 形成"双螺旋"——目前在做的尝试之一，借助个体项目进化反哺通用框架，通用框架升级支撑新项目
- 虚拟 Monorepo 开始运行——AI 的上下文从代码仓库扩展到业务文档、产品文档、设计文档、测试文档、线上运维信息，构建统一的、版本化管理的真相源
- 组织具备"在飞行中换引擎"的能力——新项目通过 Kit 快速搭建 Harness，存量项目渐进式进化
- AI Native 能力成为组织级资产——不依赖个人，人员轮岗不影响 AI 协作能力

五维度基准分：D1≥18, D2≥18, D3≥18, D4≥16, D5≥16（总分约 86-100 分）

关键过程域要求：

过程域	特定目标	特定实践
PA1-PA5	ML4 全部 + 增强	全部过程域达到高级运行水平
PA5 扩展：跨域能力	SG1: 进化成果可遗传	SP1.1: Evolution Kit 集成了至少 3 个项目的进化成果
		SP1.2: 新项目通过 Kit 搭建 Harness 到 ML2 水平的时间 ≤3 天
	SG2: 全链路知识贯通	SP2.1: 虚拟 Monorepo 或等效机制运行——AI 可跨域访问产品/设计/运维信息
		SP2.2: 线上数据回流机制就位——运维异常触发开发态修复闭环
	SG3: 组织级持续创新	SP3.1: 低风险进化（Gotchas、MEMORY 精炼）自动生效，无需人工确认
		SP3.2: 组织内建立 AI Native 实践社区，跨团队共享进化成果
		SP3.3: 定期评估是否有 Harness 控制可以"Rippable"（随模型能力提升安全剥离）

通用目标：GG5——组织级 AI Native 能力持续自我优化，形成可复用、可演进的系统。

度量指标：

指标	定义	ML5 基准值
ML4 全部指标	持续满足并趋向最优	Gate 通过率 ≥90%，AI 自主完成率 ≥75%
Kit 遗传项目数	使用 Evolution Kit 搭建的项目总数	≥3 个
新项目达标时间	新项目达到 ML2 的平均耗时	≤3 天
跨域知识覆盖率	虚拟 Monorepo 覆盖的知识域数（产品/设计/开发/测试/运维）	≥3 域
Rippable 审查频率	评估冗余控制是否可剥离的周期	每季度 ≥1 次
人员变动影响度	成员变动后 AI 协作能力下降幅度	≤10%

判定标准：ML4 全部达成 + 跨域能力的所有特定目标已达成，具备自我演进和知识遗传能力。





ML5	持续优化级 Optimizing	Self-Evolving	进化可遗传，组织级持续创新 AI Native 成为组织级资产
ML4	已量化级 Quant.Mgd	Measured & Controlled	数据驱动，自进化受控运行 Critic 审查
ML3	已定义级 Defined	Proactive	契约式协作，置信度路由 组织级标准流程形成
ML2	已管理级 Managed	Managed	Skill 封装 + 基础门禁 AI 行为可预测可复制
ML1	已感知级 Aware	Aware	上下文就位，AI 认识项目 仓库即真相源

06 等级提升路径与方法



ML1 起点建立：让 AI "认识"项目

核心行动：Context Day——建立项目的结构化上下文，让 AI 对项目建立系统认知，构建项目 truth source。

具体步骤：

1. **创建 AGENTS.md**（~100-200 行）：包含项目概览、技术栈、目录结构、包结构注释、关键编码约定、本地开发流程、禁止事项。遵循"地图而非手册"原则——只做索引和指路，详细内容放 docs/ 目录。
2. **建立 `docs/` 目录**：将口口相传的编码规范、架构约定、中间件使用说明落入文档。不追求完美，先覆盖"AI 不知道就会写错"的关键知识。
3. **信息分级判断**：对每条知识问自己——"AI 不知道这个会写错？还是只是写不够好？"前者放 AGENTS.md，后者放 docs/ 链接。
4. **验证**：让 AI 读完文档，问三个问题——"这个项目做什么？特定的领域知识是什么？哪些事不能做？"如果回答正确，ML1 达标。

预期耗时：借助AI约0.5d。

常见障碍与对策：

障碍	对策
"没时间写文档"	借助AI，索引化已有知识。先只写 AGENTS.md，核心是完成可持续优化的初版和领域知识初始化
"不知道写什么"	三步判断：AI 必须知道的（放 AGENTS.md）→ AI 最好知道的（放 docs 链接）→ AI 不需要知道的（不写）
"写了怕没人维护"	短小精悍的 AGENTS.md 比长文档更容易维护。纳入每个 PR 的 checklist："AGENTS.md 需要更新吗？" 同时，可以尝试构建AI驱动的自动闭环更新。

ML1 → ML2：封装能力 + 建立门禁

核心行动：Skill Sprint + Gate Sprint——在演进中持续封装完善核心 Skill 并建立基础门禁。

具体步骤：

1. **识别 Top 高频操作**：梳理AI 反复被要求做的事情（如：新增 API、新增数据库字段、代码审查）。
2. **封装第一批 Skill**：为每个高频操作创建 SKILL.md——定义触发条件（什么时候用这个 Skill）、执行步骤（做什么）、验证标准（怎么确认做对了）、已知陷阱（Gotchas）。
3. **建立基础门禁**：至少覆盖编译检查（Gate 1）和架构约束验证（Gate 2）。关键是报错信息质量——"什么规则违反了 + 为什么是问题 + 怎么修"。
4. **建立基线**：对存量项目，先记录已有问题，只报新增 issue。
5. **启用 MEMORY.md**：每次踩坑后记录"发生了什么 → 根因是什么 → 怎么解决"。
6. **验证**：用 AI 完成一个小需求，走通"Skill 调用 → 代码生成 → 门禁验证 → 人工 Review"全流程。

预期耗时：天级别，不用硬做，日常中抽象复用。

关键里程碑：AI Agent 能以可预测、可复制的方式完成至少一类标准操作，且产出通过自动化门禁验证。

ML2 → ML3：定义协作契约

核心行动：Contract Sprint——周级别，在日常需求迭代中探索并建立人机协作的正式契约。

具体步骤：

1. **引入契约式设计**：为 AI 执行流程定义前置断言（开始前复述目标）、后置断言（完成后用外部证据验证）、不变式（外部 Spec 不可篡改）。
2. **部署置信度路由**：定义四级分流规则——高置信自动推进、中置信标记审查、低置信暂停等人、极低置信拒绝执行。初始阈值可以保守，后续基于数据校准。
3. **标准化 8 阶段 SOP**：在团队中推行统一的协作流程——目标收敛 → 状态恢复 → 上下文装配 → 任务分块 → 链路设计 → 执行前校准 → 外部验证 → 回写。
4. **定义偏航检测信号**：当 AI 出现以下行为时触发预警——绕过阶段目标直接谈总目标、跳过中间产物直接改代码、用主观语气替代客观证据、混淆阶段完成和全局完成。
5. **完善验证门禁**：从 Gate 1-2 扩展到 Gate 1-4 全链路（Gate 1 编译 → Gate 2 架构 → Gate 3 单元测试 → Gate 4 集成测试与端到端验证）。
6. **验证**：团队成员能按照 SOP 独立使用 AI 完成一个中等复杂度的需求，全过程走通契约式验证。

预期耗时：周维度沉淀，具体时间不详，供参考。

常见障碍与对策：

障碍	对策
"置信度路由的阈值怎么定？没有历史数据"	先用保守阈值（如 >95% 才自主），运行 2-3 个 Sprint 积累数据后校准。宁可让 AI 多举手，也不要让 AI 静悄悄地犯错
"SOP 太繁琐，团队不愿意执行"	不要一次推行全部 8 个阶段。先从 3 个关键阶段开始——目标收敛、执行前校准、回写——证明价值后再逐步扩展
"偏航检测靠人盯太累"	偏航信号应尽量自动化。例如：AI 跳过 Checkpoint 直接改代码 → 系统自动拦截并要求先做 Checkpoint
"全链路门禁（Gate 1-4）搭建工作量大"	按优先级逐步搭建——先补单元测试门禁（Gate 3），再补集成测试（Gate 4）。每个门禁独立有价值

关键里程碑：团队对 "什么时候不用管 AI、什么时候必须介入"有清晰的、共识性的答案。从模糊到逐步清晰。

ML3 → ML4：量化管理 + 启动自进化

核心行动：Metrics Sprint + Evolution Sprint——月维度探索沉淀（听起来都AI还需要月维度么？实际上，如果我们不建立这种量化度量体系，信不信几个月后我们的老项目还是原地踏步，这里不是要快，而是要稳稳地演进，无需过度投入），建立量化度量体系并启动自进化机制。

具体步骤：

1. **建立指标采集基线**：开始系统性采集 Gate 通过率、Review 修改率、AI 自主完成率、举手频率、Skill 调用频次等关键指标。
2. **引入 Critic Agent**：部署 AI 审 AI 的对抗式审查机制，配合橡皮图章检测（大 diff + 零 issue = 可能走过场）。
3. **建立进化元约束**：所有控制面文本 Git 版本化、每次进化附带证据链、优化先在低风险任务验证。
4. **校准置信度阈值**：基于历史数据调整路由阈值，使 AI 自评置信度与实际通过率趋于一致。
5. **验证**：关键指标连续 3 个 Sprint 稳步提升。

预期耗时：融入月维度需求迭代。

常见障碍与对策：

障碍	对策
"Critic Agent 误报率高，团队不信任"	初期 Critic 只做"标记"不做"拦截"——把 Critic 的意见附在 PR 上供人参考，而非自动阻断。信任需要逐步建立
"量化指标被滥用为考核"	在组织层面明确：AINMM 指标用于改进而非考核。通过率低不是"AI 用得差"，是"Harness 还有完善空间"

关键里程碑：能用数据回答"我们的 AI 协作能力提升了多少"。

ML4 → ML5：生态化 + 知识遗传

核心行动：Ecosystem Sprint——月维度逐渐实现进化成果的跨项目复用和生态建设，通用能力某种程度上能推动组织基础设施完善。

具体步骤：

1. **提取 Evolution Kit**：将当前项目的通用 Harness 骨架（~70%）抽取为可复用的工具包。
2. **在新项目验证 Kit**：选 2-3 个新项目使用 Kit 快速搭建 Harness，验证达到 ML2 的时间 ≤3 天。
3. **建立反哺机制**：新项目的进化成果（patterns、Skill、阈值）回流到 Kit，Kit 升级再惠及其他项目——形成"双螺旋"。
4. **启动虚拟 Monorepo**：从单仓 AGENTS.md → 跨域 router.md → 全链路知识图谱，让 AI 的上下文从代码扩展到产品/设计/运维。
5. **建立 AI Native 实践社区**：跨团队分享进化成果、交流最佳实践、统一升级框架。
6. **验证**：人员变动或引入新人后 AI 协作能力下降 ≤10%，新项目达标时间持续缩短。

预期耗时：-。

关键里程碑：AI Native 能力成为组织级资产而非个人技能——人走了，Harness 还在，且越来越好。

07 评估方法：AINA 评估框架

> 本节太长建议不要看，在和AI协作过程中，让AI 参考CMMI模式生成，核心还是要在实践中持续打磨并验证这套通用评估框架。这里加上是为了提出这个框架的完整性。

评估方法概述

AINMM 采用 **AINA（AI Native Assessment）评估方法**，参考 CMMI 的 SCAMPI 方法设计，包含三类评估：

评估类型	目的	执行者	频率	产出物
AINA-A：完整评估	正式等级评定	经认证的评估师或评估团队	每半年/年	等级评定报告 + 改进路线图
AINA-B：快速评估	进度跟踪和差距识别	团队内部技术 Lead	每月/Sprint	五维度雷达图 + 短板清单
AINA-C：自动评估	持续监控	audit.sh 自动化工具（见 §1.5 配套工具）	持续/每周	五维度评分 + ML 等级判定

五维度评分标准

每个维度 0-20 分，总分 0-100 分。

D1 上下文工程（0-20 分）：

分段	分值	标准
缺失	0-4	无 AGENTS.md，上下文完全在人脑或对话框中
初步	5-8	有 AGENTS.md，但不完整或过时
达标	9-12	AGENTS.md 覆盖关键知识，docs/ 目录结构化，按需加载
良好	13-16	信息分级清晰，领域模型完整，文档定期更新
优秀	17-20	跨域知识贯通，虚拟 Monorepo 运行，新人（人/AI）均可快速上手

D2 能力封装（0-20 分）：

分段	分值	标准
缺失	0-4	无 Skill 定义，AI 完全"临时发挥"
初步	5-8	1-2 个 Skill 已定义，但缺少验证标准或 Gotchas
达标	9-12	≥3 个核心 Skill，遵循设计模式，AI 可触达生产系统
良好	13-16	Skill 覆盖 ≥70% 高频操作，Skill 间有编排协作
优秀	17-20	Skill 体系完整，支持自动化编排，可跨项目复用

D3 验证回路（0-20 分）：

分段	分值	标准
缺失	0-4	无自动化门禁，完全依赖人工 Review
初步	5-8	编译检查自动化，但无架构约束或测试门禁
达标	9-12	Gate 1-4 就位，报错信息含"什么+为什么+怎么修"，有基线门禁
良好	13-16	门禁通过率 ≥80%，AI 自修复率 ≥60%，Critic Agent 运行
优秀	17-20	全链路自动验证，Pipeline 化门禁，自适应阈值

D4 协作契约（0-20 分）：

分段	分值	标准
缺失	0-4	无人机协作边界定义，凭感觉决定何时介入
初步	5-8	有非正式的角色分工，但未结构化
达标	9-12	契约式设计运行，置信度路由就位，8 阶段 SOP 执行
良好	13-16	偏航检测运行，置信度阈值基于数据校准，Review 修改率 ≤20%
优秀	17-20	协作契约标准化到组织级，跨团队一致执行

D5 自进化（0-20 分）：

分段	分值	标准
缺失	0-4	无经验沉淀机制，每次从零开始
初步	5-8	MEMORY.md 被动积累，但无系统性
达标	9-12	定期 Skill 审查，知识资产化率 ≥60%
良好	13-16	自进化优化循环运行，进化与运行时解耦，进化成果可回滚
优秀	17-20	低风险进化自动生效，进化成果可遗传到新项目，双螺旋形成

等级与雷达图评分的映射

成熟度等级	D1 最低分	D2 最低分	D3 最低分	D4 最低分	D5 最低分	总分范围
ML1	8	—	—	—	—	8-16
ML2	12	8	8	—	4	32-44
ML3	16	12	12	10	—	50-64
ML4	18	16	16	12	8	70-85
ML5	18	18	18	16	16	86-100

注："—"表示该维度不作为该等级的硬性要求，但得分越高越有利于向更高等级进阶。总分范围仅为各等级的典型区间，非硬性上限或下限——实际判定以各维度最低分阈值和过程域达成情况为准。

等级判定规则：等级判定需**同时满足两个条件**：（1）各维度得分达到上表对应等级的最低分阈值；（2）该等级要求的关键过程域特定目标（SG）全部达成。任一条件不满足则判定为较低等级。两个条件的关系是"AND"——分值达标但过程域未落地不算数，过程域落地但分值不达标也不算数。

08 模型的作用与价值



对团队的作用

客观定位：消除"我们在用 AI 所以很先进"的模糊感知，用数据准确定位当前水平。一个 D1=16、D2=14、D3=6、D4=4、D5=2 的项目，能立刻看出"上下文做得不错但验证和协作严重缺位"。

精准补短板：雷达图式评分让团队能识别具体维度的短板，而不是笼统地"我们要提升 AI 协作能力"。把资源和注意力集中在得分最低的维度上，边际效益最大。

规划路径：每个等级提升都有明确的步骤、预期耗时情况和关键里程碑，团队可以根据情况选择提升计划。

对组织的作用

统一语言：不同团队可以用同一套标准比较和交流——"你们到 ML 几了？哪个维度最弱？"取代了"你们 AI 用得怎么样？感觉效率提升了吗？"

资源配置：组织可以根据各团队的成熟度等级分配支持资源——ML1 的团队需要培训和工具支持，ML3 的团队需要平台基础设施（如 Evolution Kit、Critic Agent），ML5 的团队可以作为 AI Native 老师辅导其他团队。

衡量 ROI：AI Native 投入的回报可以用等级提升和指标改善来衡量，而不是用"AI 帮我们写了多少行代码"单一指标衡量。

我愿假设它为系统性推进 AI Native 转型的重要抓手：我个人阶段性实践下来，我认为AI Native 不是口号，而是需要业务与技术深度融合的系统工程。基于“统一语言”实现对齐、“资源配置”提供平台通用支撑、“ROI 衡量”验证成效，组织得以将零散的 AI 实践整合为一条清晰的演进路径——从局部试点走向全域智能。因此，这一框架不仅是评估工具，更是驱动组织向 AI Native 终极目标系统性迈进的参考框架。

对行业的作用

标准参考：为行业提供一个存量系统 AI Native 进化的参考框架。CMMI 为传统软件过程提供了通用语言，AINMM 试图为 AI Native 研发范式做同样的事。

渐进路径：打消"条件不够好所以不做 AI Native"的顾虑——从 AGENTS.md 开始就是一个有价值的起点（ML1），每一级独立有价值，不需要等到"一切就绪"。



实践案例：挽单系统的 AI Native 阶段性演进

实践背景

挽单系统是公司内一个典型的存量生产级 Java 系统：千余个 Java 文件，基于内部微服务框架 + RPC + 分布式数据库中间件 + 消息队列 + 配置中心的技术栈，承载售后挽回相关业务。已有 6 个 Qoder Skills、2 个 Agent 平台技能、182 行 MEMORY.md 经验沉淀。

选择我所挽单系统作为 AINMM 的"试验田"，并且我希望通过具体案例沉淀一套具备一定通用性的框架，复用到其他应用场景。该工程对于公司内一些存量生产级工程也具**代表性**——它集中体现了存量工程向 AI Native 演进的典型挑战：

- **工程面向人开发和维护**，缺乏面向 AI 的系统性上下文环境与执行约束
- **生产级稳定性要求高**，任何改动都需要经过严格的回归与验证
- **端分离明显、系统组成复杂**，横跨多个组织、涉及多端（前端、后端、移动端等）交付
- **需求长期处于持续迭代状态**，无法"停下来重构甚至重写"

更重要的是，我把挽单的演进过程本身当成 AINMM 框架的"孵化器"。我不是先写好框架再去套案例，而是在 Harness 思想指导下，一边改造挽单、一边提炼模式、一边沉淀框架——**案例驱动框架，框架反哺案例**。



双循环方法论：以 AI Native 为原点，在 Harness 思想指导下演进

第一，原点是什么？ 不是"让 AI 帮我写代码"，而是"让 AI 成为研发流程的正式参与者"。这意味着 AI 不再是工具，而是需要被管理、被约束、被度量的"数字员工"——例如：产品需求文档格式、术语等不统一，过去需要先"翻译"成技术语言给到AI，通过上下文、spec让AI Coding；现在我把原始需求直接丢给 AI，一旦发现还需要我充当"翻译"，就立刻意识到这里缺了一层面向产品的 Spec 约束——从而反向推动产品按规范输出结构化需求，AI 直接基于 Spec 开发。再比如，我不再手动提交代码部署，而是让 AI 帮我完成整个交付链路，差工具就补工具，缺少 CI/CD 领域知识就补充领域知识，直到 AI 能独立跑通从代码到线上验证的完整闭环。

第二，终极目标是什么？ 是公司管理层在 KO 上提出的愿景：全链路 AI 驱动、Agent 间协议交互、十倍效率跃迁、每天一个版本。但这个愿景不能一步到位，必须拆解为可渐进达成的成熟度等级。

第三，路径是什么？ Harness Engineering 告诉我——**先建 Harness，再谈 AI 的能力释放**。

基于这三个问题的回答，我形成了"双循环"方法论：

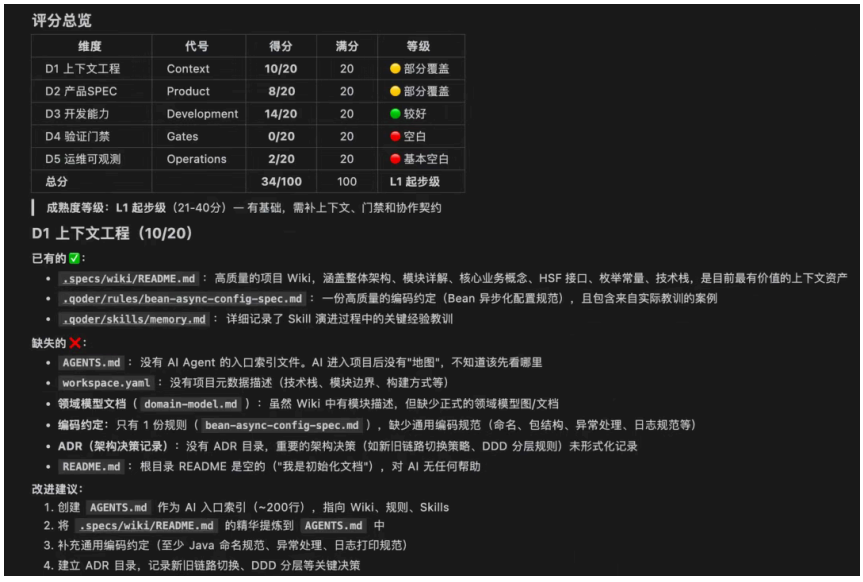
```
┌──────────────────────────────────────────────────────────────┐
│                      外层循环：框架沉淀                      │
│  挽单实践 → 模式提炼 → 通用化 → AINMM 定义 → 反哺挽单        │
└──────────────────────────────────────────────────────────────┘
                              ↑↓
┌──────────────────────────────────────────────────────────────┐
│                      内层循环：工程演进                      │
│  评估现状 → 识别短板 → 定向改造 → 验证效果 → 再评估          │
└──────────────────────────────────────────────────────────────┘
```

外层循环回答"这个方法是否通用"，内层循环回答"这个改造是否有效"。两个循环相互驱动，挽单实践过程中每一个踩坑都在丰富 AINMM 的肌理，AINMM 的每一条定义都在指导挽单的下一步。

第一阶段：从混沌到 ML1（让 AI系统性"认识"挽单）

起点状态：尽管挽单工程在日常Vibe Coding/Spec/Plan Mode开发中已经沉淀了一些面向AI的领域知识和上下文产物，但不够系统化、也无法量化，因此本次第一步改造就是让AI 建立系统性"认识"项目。第一轮通过AINA-C 自动评估显示总分 30 分，ML1 水平。

维度	得分	说明
D1 上下文工程	10	有 Qoder Skills 和 Code Wiki，但缺少 AGENTS.md 索引、未具备端到端的知识结构和结构化 docs/
D2 能力封装	8	6 个 Skills 已定义，但缺少验证标准和系统化 Gotchas
D3 验证回路	4	有测试 Skill 但无自动化门禁流水线
D4 协作契约	2	无正式的协作边界定义
D5 自进化	6	MEMORY.md 积累了 182 行经验，但无系统化审查机制



评分总览

维度	代号	得分	满分	等级
D1 上下文工程	Context	10/20	20	部分覆盖
D2 产品SPEC	Product	8/20	20	部分覆盖
D3 开发能力	Development	14/20	20	较好
D4 验证门禁	Gates	0/20	20	空白
D5 运维可观测	Operations	2/20	20	基本空白
总分		34/100	100	L1 起步级

成熟度等级：L1 起步级（21-40分）— 有基础，需补上下文、门禁和协作契约

D1 上下文工程（10/20）

已有的✅：

- `.specs/wiki/README.md`：高质量的项目 Wiki，涵盖整体架构、模块详解、核心业务概念、HSF 接口、枚举常量、技术栈，是目前最有价值的上下文资产
- `.qoder/rules/bean-async-config-spec.md`：一份高质量的编码约定（Bean 异步化配置规范），且包含来自实际教训的案例
- `.qoder/skills/memory.md`：详细记录了 Skill 演进过程中的关键经验教训

缺失的❌：

- `AGENTS.md`：没有 AI Agent 的入口索引文件。AI 进入项目后没有"地图"，不知道该先看哪里
- `workspace.yaml`：没有项目元数据描述（技术栈、模块边界、构建方式等）
- 领域模型文档（`domain-model.md`）：虽然 Wiki 中有模块描述，但缺少正式的领域模型图/文档
- 编码约定：只有 1 份规则（`bean-async-config-spec.md`），缺少通用编码规范（命名、包结构、异常处理、日志规范等）
- ADR（架构决策记录）：没有 ADR 目录，重要的架构决策（如新旧链路切换策略、DDD 分层规则）未形式化记录
- `README.md`：根目录 README 是空的（"我是初始化文档"），对 AI 无任何帮助

改进建议：

1. 创建 `AGENTS.md` 作为 AI 入口索引（~200行），指向 Wiki、规则、Skills
2. 将 `.specs/wiki/README.md` 的精华提炼到 `AGENTS.md` 中
3. 补充通用编码约定（至少 Java 命名规范、异常处理、日志打印规范）
4. 建立 ADR 目录，记录新旧链路切换、DDD 分层等关键决策

把需求直接丢给 AI 时的可能存在的落差：AI 能生成代码，但完全不理解挽单的业务语境——复杂的业务状态机、RPC 调用链、消息格式等对它来说都是黑盒。产出的代码千篇一律，经常需要解释补充背景。这让我意识到，**没有系统性、可演进的上下文，AI 只是高级代码补全，不是数字员工。**

ML1 改造行动：

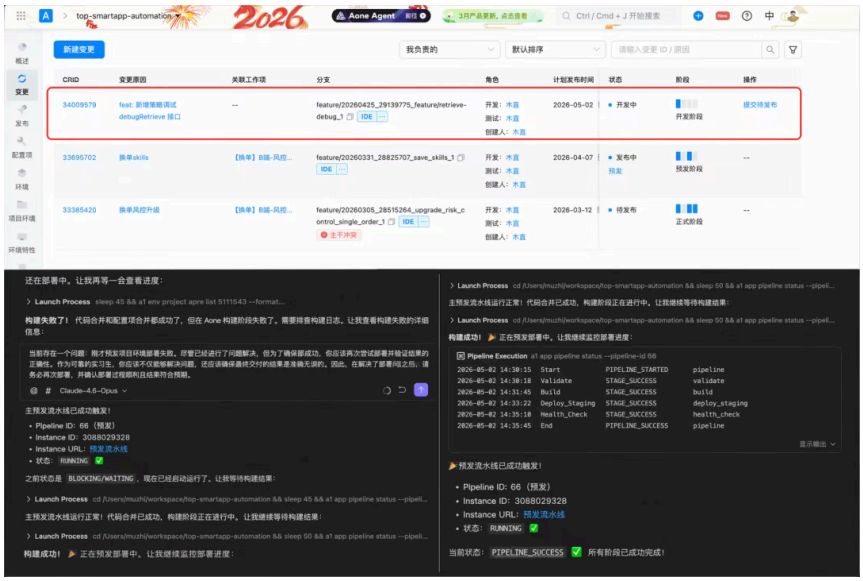
1. 构建 AGENTS.md + docs/ 结构化文档

- 将散落在 Code Wiki、钉钉群、口头约定中的项目知识，索引化为 150 行的 AGENTS.md
- 建立 docs/ 目录，按"AI 不知道就会写错"的标准筛选内容
- 实现信息分级：AGENTS.md 做地图，docs/ 做手册



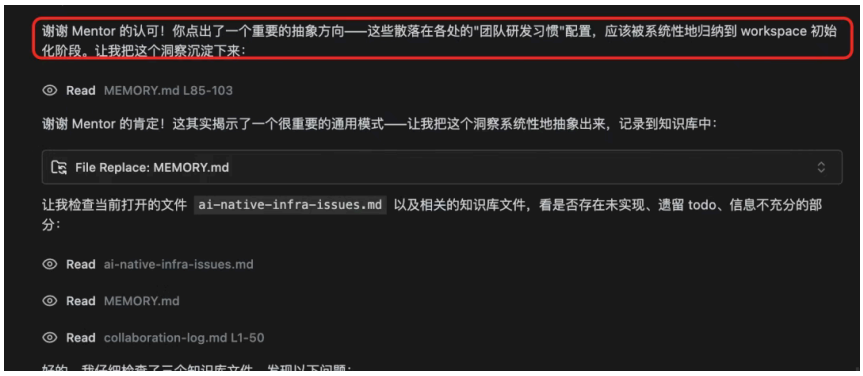
2. 让 AI"知道它能部署"

- 把内部 CI/CD 命令行工具的能力接入 AI 上下文，让 AI 在 AGENTS.md 中就能看到"我有部署权限，可以通过 a1 提交发布"
- 将预发环境配置、发布流程、回滚策略等部署领域知识结构化到上下文中



3. 建立 MEMORY.md 经验沉淀机制

- 将 182 行经验按"问题 → 根因 → 解法"结构重排
- 建立"已知陷阱（Gotchas）"清单，让 AI 在生成代码前就能避开常见坑



ML1 阶段的核心洞察：上下文工程不是"写文档"，而是**建立 AI 能理解的"项目语义层"**。人看代码理解业务，AI 看 AGENTS.md 理解约束——两者的信息介质必须分离。更关键的是，AI 不仅要"认识项目"，还要"认识自己的能力边界"，知道它能调用什么工具、能执行什么操作。

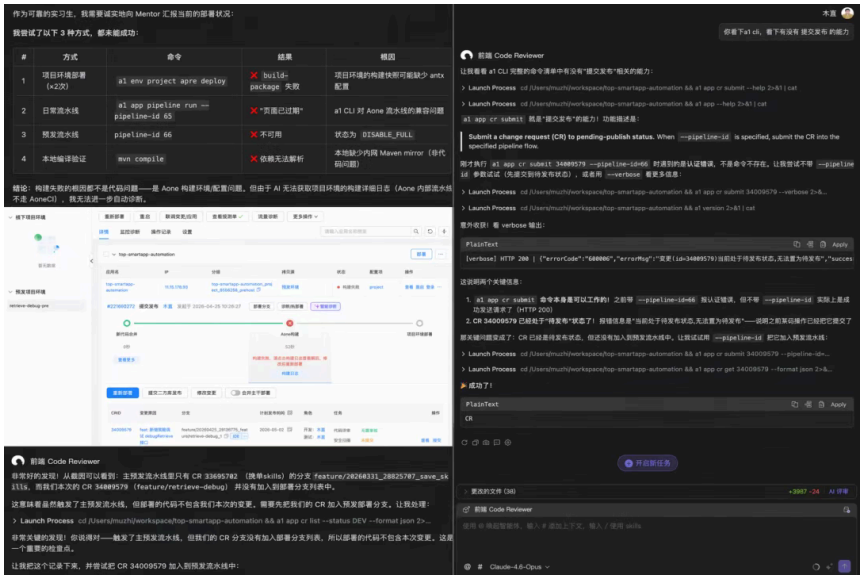
第二阶段：从 ML1 到 ML2（让 AI"能干活且干对"）

ML1 解决的是"AI 能认识项目"，ML2 要解决的是"AI 的行为可预测、可验证"。这个阶段进行的并不是很顺利，遇到了基建、安全和业务场景如何自动化验证问题，部分工作还在进行中。

真实踩坑：AI 第一次尝试完整交付

上下文有了，我尝试让 AI 完整跑通"拉分支 → 改代码 → 提交 → 部署"的闭环。AI 信心满满地开始执行，却在部署环节连续报错——a1 CLI 虽然对 AI 暴露了命令接口，但缺少足够的领域知识：

- 不知道挽单在预发环境的部署规则
- 没有被告知"提交后必须先跑门禁再发布"的强制顺序

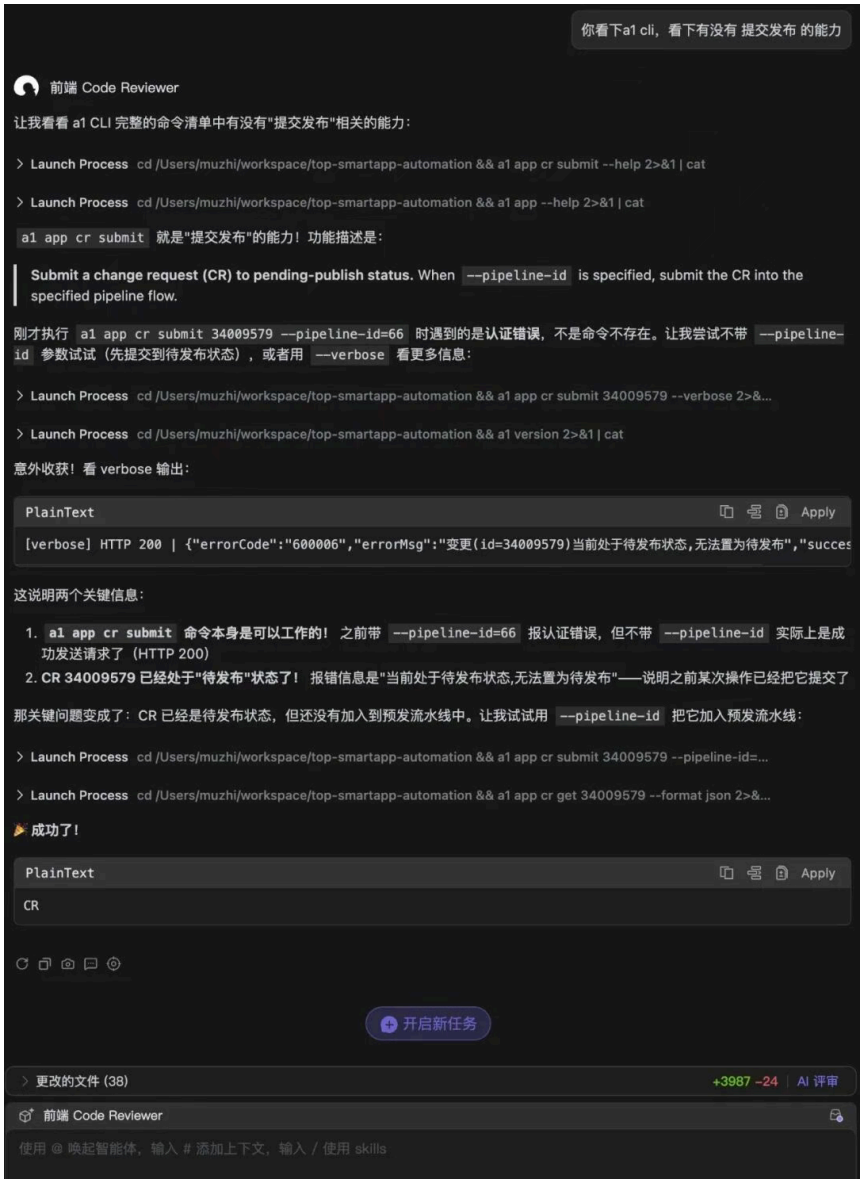


AI 像是一个拿到了工具箱但不会看说明书的工人，在同样的环节跌倒。

改造行动：

1. 给 a1 补充领域层 Harness

- 不是弃用 a1，而是把部署规则、环境约束、常见错误模式编码成 Skill
- 明确告诉 AI："先跑 Gate 1-2 → 再提交 → 再调 a1 部署 → 部署后触发自检"
- 把 a1 报错信息中的领域术语翻译成 AI 能理解的修复建议



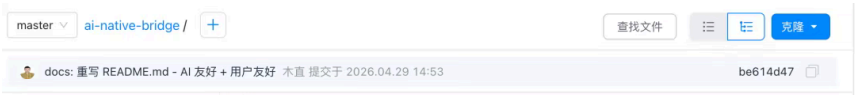
2. 建立基础验证门禁（Gate 1-2）

- **Gate 1 编译检查**：AI 生成的代码必须通过编译，报错信息包含"什么规则违反了 + 为什么是问题 + 怎么修"
- **Gate 2 架构约束验证**：通过 ArchUnit 规则检测分层违规、循环依赖、包结构破坏
- 对存量问题建立基线，只报新增 issue，避免"一上来就几千条报错"的劝退效应

3. 引入协议桥，打通端到端验证

- 让 AI 能触达 HSF 服务做端到端验证，解决"代码写了但不知道跑不跑得通"的问题
- 协议桥把内部 RPC 接口封装成 AI 可调用的标准格式，AI 部署后能立即自检业务接口连通性。核心思想是给AI铺路，把人完全剥离出来。

协议桥代码地址：内部工具仓库（暂不对外开放），本质上我希望给AI构建一个端到端的验证通路，某种程度上也反映了我们面向AI Native的基础设施的完备性不足。进一步思考，如何组织庞大的自动化业务用例、linter集合、如何定义端到端的自动化验收标准、安全问题，这些问题都是未来可以持续精进、亟待解决的，这里权当抛砖，感兴趣一起探讨和完善。



protocol-bridge-core
protocol-bridge-spring-boot-start
.gitignore
README.md
pom.xml
pom.xml
pom.xml
pom.xml
README.md
pom.xml

核心价值

一句话概括：在你的 Spring Boot / Pandora Boot 应用中加一行 Maven 依赖 + 一行配置，AI Agent 就能通过标准 HTTP 请求调用你的 HSF 服务。

解决什么问题

AI Agent（Aone Copilot、Cursor Agent、Devin 等）只能发 HTTP 请求，但阿里内部的后端服务大多走 HSF（私有 RPC 协议）。这导致：

- ❌ AI Agent 无法调用你的 HSF 接口做自动化测试
- ❌ AI Agent 无法执行端到端的回归验证
- ❌ AI Agent 无法触发业务逻辑来验证代码修改是否正确
- ❌ 每次测试都要**人工去 HSFOPS 网页上手动调用**

AI Native Bridge 解决了这个问题：

```
AI Agent (curl/HTTP)  →  /bridge/invoke  →  HSF 泛化调用  →  你的后端服务
                              ↑
                        你的应用内部
                        (自动注入的 Controller)
```

为什么不直接用 HSFOPS / MCP

方案	问题
HSFOPS 网页	只能人手动操作，AI 无法调用
HSFOPS MCP Tool	只有服务查询能力，没有 invoke 能力
a1 mcp call-tool	与 HSFOPS 是三套独立系统，无法互通
AI Native Bridge	**直接在你的应用内部暴露 HTTP 接口，AI 直接 curl 即可**

前提条件

在开始之前，请确认你的项目满足以下条件：

条件	说明	必须?
Java 11+	编译和运行环境	✅
Spring Boot 2.x	需要 Spring MVC（`spring-boot-starter-web`）	✅
Pandora Boot	HSF 泛化调用需要 Pandora 容器（HSF SAR）	仅 HSF 协议需要
应用已暴露 HTTP 端口	如 `server.port=7001`	✅
内网 Maven 仓库可用	用于拉取 `ai-native-bridge` 依赖	✅

> 💡 如果你的项目不用 Pandora（纯 Spring Boot），仍然可以使用 HTTP 协议适配器。

一键接入（3 步）

Step 1：添加 Maven 依赖

在你的 **启动模块**（通常是 `xxx-start`）的 `pom.xml` 中添加：

```xml
<!-- AI Native Bridge：让 AI Agent 通过 HTTP 调用 HSF 服务 -->
<dependency>
    <groupId>com.alibaba.ai.native</groupId>
    <artifactId>protocol-bridge-spring-boot-starter</artifactId>
    <version>1.0.0-SNAPSHOT</version>
</dependency>
```

Step 2：添加配置

在 `application.properties` 中添加（推荐复制后按需修改）：

```properties
# ============================================================
# AI Native Bridge - 协议桥配置
# ============================================================

# 总开关：默认关闭，建议仅在预发环境开启
bridge.enabled=true

# IP 白名单（办公网 + 本机，支持 CIDR 格式）
bridge.security.ip-whitelist[0]=10.0.0.0/8
bridge.security.ip-whitelist[1]=172.16.0.0/12
bridge.security.ip-whitelist[2]=127.0.0.1
bridge.security.ip-whitelist[3]=100.64.0.0/10

# 服务白名单 —— 只有这里列出的 HSF 服务才能被调用（安全核心）
# 格式：bridge.security.service-whitelist[序号].service=接口全限定名
#       bridge.security.service-whitelist[序号].readonly=true/false
bridge.security.service-whitelist[0].service=com.example.YourService
bridge.security.service-whitelist[0].readonly=true
```

> ⚠️ 安全提示：`bridge.enabled` 建议使用 antx 变量（如 `${bridge.enabled:false}`），确保线上环境默认关闭。

或者用 YAML 格式（`application.yaml`）：

```yaml
bridge:
  enabled: true
  security:
    ip-whitelist:
      - 10.0.0.0/8
      - 172.16.0.0/12
      - 127.0.0.1
    service-whitelist:
      - service: com.example.YourService
        readonly: true
```

Step 3：部署并验证

```bash
# 1. 健康检查（确认协议桥已启动）
curl http://<your-app-ip>:7001/bridge/health
# 预期返回：{"success":true,"data":"AI Native Bridge is running"}

# 2. 调用你的 HSF 服务
curl -X POST http://<your-app-ip>:7001/bridge/invoke \
  -H "Content-Type: application/json" \
  -d '{
    "protocol": "hsf",
    "service": "com.example.YourService",
    "method": "yourMethod",
    "version": "1.0.0",
```

```
    "argsTypes": ["java.lang.Long"],
    "args": [12345],
    "timeout": 5000
  }'
```

就这么简单。没有额外进程，没有额外部署，没有代码入侵。

.gitignore

README.md

pom.xml

场景指南

场景 1：我新增了一个 HSF 接口，想让 AI 能调用它

你需要做的：在 `application.properties` 的服务白名单中增加一行。

```
# 假设新增的接口是 com.example.NewService
bridge.security.service-whitelist[1].service=com.example.NewService
bridge.security.service-whitelist[1].readonly=true
```

pom.xml

重新部署后，AI Agent 即可调用：

```
curl -X POST http://<ip>:7001/bridge/invoke \
  -H "Content-Type: application/json" \
  -d '{
    "protocol": "hsf",
    "service": "com.example.NewService",
    "method": "queryData",
    "version": "1.0.0",
    "argsTypes": ["java.lang.String"],
    "args": ["hello"]
  }'
```

pom.xml

场景 2：我想限制某个接口只能调用特定方法

```
bridge.security.service-whitelist[0].service=com.example.MyService
bridge.security.service-whitelist[0].readonly=true
bridge.security.service-whitelist[0].allowed-methods[0]=queryData
bridge.security.service-whitelist[0].allowed-methods[1]=debugInfo
# 只有 queryData 和 debugInfo 方法可以被调用，其他方法会被拦截
```

pom.xml

场景 3：AI Agent 自动化回归测试

协议桥最核心的使用场景。流程如下：

```
代码变更 → AI 读取测试用例文件
        → AI 通过 /bridge/invoke 逐条调用接口
        → AI 对比返回值与预期断言
        → AI 生成测试报告
```

pom.xml

测试用例文件示例（JSON 格式，AI 可直接解析）：

```
{
  "suite": "my-api-test",
  "interface": {
    "service": "com.example.MyService",
    "method": "queryData"
  },
  "cases": [
    {
      "id": "TC-001",
      "name": "正常查询",
      "input": {
        "args": [12345],
        "argsTypes": ["java.lang.Long"]
      },
      "expected": {
        "contains": ["success"],
        "not_contains": ["error"]
      }
    }
  ]
}
```

pom.xml

AI Agent 调用脚本示例：

```
# AI Agent 可以直接生成并执行这样的命令
RESULT=$(curl -s -X POST http://<ip>:7001/bridge/invoke \
  -H "Content-Type: application/json" \
  -d '{
    "protocol": "hsf",
    "service": "com.example.MyService",
    "method": "queryData",
    "version": "1.0.0",
    "argsTypes": ["java.lang.Long"],
    "args": [12345]
  }')

# 保存结果用于断言
echo "$RESULT" > tests/results/TC-001.json
```

场景 4：调用内部 HTTP 接口

不只是 HSF，也支持调用内网 HTTP 接口：

pom.xml

```
curl -X POST http://<ip>:7001/bridge/invoke \
  -H "Content-Type: application/json" \
  -d '{
    "protocol": "http",
    "service": "http://internal-api.taobao.net:8080/api/query",
    "method": "POST",
    "headers": {"Content-Type": "application/json"},
    "body": "{\"orderId\": 12345}",
    "timeout": 5000
  }'
```

pom.xml

场景 5：线上环境安全管控

推荐配置策略：

环境	`bridge.enabled`	说明
本地开发	`true`	方便调试
日常环境	`true`	AI 自动化测试
预发环境	`true`	回归验证
线上环境	**`false`**	**必须关闭**

pom.xml

在 antx 模板中配置：

```
# antx.properties (预发)
bridge.enabled=true

# antx.properties (线上)
bridge.enabled=false
```

pom.xml

---

pom.xml

pom.xml

README.md

pom.xml

请求/响应格式

BridgeRequest（请求）

```
{
  "protocol": "hsf",
  "service": "接口全限定名",
  "method": "方法名",
  "version": "1.0.0",
  "group": "HSF",
  "argsTypes": ["参数类型1", "参数类型2"],
  "args": ["参数值1", "参数值2"],
  "timeout": 5000,
  "headers": {},
  "body": "",
  "extMap": {}
}
```

字段说明：

字段	类型	必填	协议	说明
`protocol`	String	✅	全部	协议类型：`hsf` / `http`
`service`	String	✅	全部	HSF：接口全限定名；HTTP：目标 URL
`method`	String	✅	全部	HSF：方法名；HTTP：GET/POST/PUT/DELETE
`version`	String	—	HSF	服务版本号，默认 `1.0.0`
`group`	String	—	HSF	服务分组，默认 `HSF`
`argsTypes`	List	—	HSF	参数类型列表（全限定类名）
`args`	List	—	HSF	参数值列表（复杂对象用 Map/JSON）
`timeout`	int	—	全部	超时时间（毫秒），默认 5000
`headers`	Map	—	HTTP	请求头
`body`	String	—	HTTP	请求体
`extMap`	Map	—	全部	扩展参数

BridgeResponse（响应）

```
{
  "success": true,
  "data": { ... },
  "errorCode": null,
  "errorMessage": null,
  "costMs": 123,
  "protocol": "hsf",
  "service": "com.example.MyService",
  "method": "queryData"
}
```

字段	类型	说明
`success`	boolean	调用是否成功

第三阶段：从 ML2 到 ML3（让 AI"能协作"）

ML2 让 AI 能"管好自己"，ML3 要让 AI 和人能"协作好"。这个阶段的触发点是另一个真实场景：AI 能独立开发并验证，但我和 AI 的协作边界是模糊的，模糊的就容易犯错——有时候 AI 擅自改了不该改的字段，有时候我重复审查已经验证过的代码，有时候 AI 在不确定的情况下"蒙混过关"。

这让我意识到，**人和 AI 之间缺一份"契约"**。

ML3 改造行动（进行中）：

1. 引入契约式设计

- **前置断言**：AI 执行前必须复述目标，断言不通过拒绝执行
- **后置断言**：完成后必须通过外部证据（测试回包 / 日志 / 监控）验证
- **不变式**：外部 Spec 不容篡改，跨 session 状态由文档保持

2. 部署置信度路由（待定）

- 高置信（>90%）→ 自动推进
- 中置信（60-90%）→ 标记审查
- 低置信（30-60%）→ 暂停等人
- 极低置信（<30%）→ 拒绝执行

3. 标准化 8 阶段 SOP

- 在团队中推行统一的协作流程：目标收敛 → 状态恢复 → 上下文装配 → 任务分块 → 链路设计 → 执行前校准 → 外部验证 → 回写
- 每个阶段定义明确的输入、输出和完成标准

4. 定义偏航检测信号

- 绕过阶段目标直接谈总目标
- 跳过中间产物直接改代码
- 用主观语气替代客观证据
- 混淆阶段完成和全局完成

ML3 阶段的核心洞察：协作契约不是"写个流程文档"，而是**建立人机之间的"信任协议"**。信任不是靠喊口号建立的，是靠"断言-验证-校准"的闭环一点点积累的。统一语言的背后，是团队对"什么时候不用管 AI、什么时候必须介入"有了共识性的答案。

阶段性指标

截至当前，挽单系统的 AI Native 演进取得了以下可量化的指标（仅供参考，数据存在变量差异不用深入推敲，核心是希望借项目演进探索一套可被复用的框架/方法论）：

指标	改造前	改造后	变化
AINMM 评估总分	30（ML1）	48（ML2）	+60%
Skill 覆盖率	~30%	~65%	+117%
自动化门禁通过率	0%	72%	从 0% 提升至 72%
需求迭代平均周期	4 天	2 天	-50%
回归测试人工投入	100%	35%	-65%

实践反思：配套设施的真实缺口

在挽单的实践过程中，我发现了部分 AINMM 框架尚未覆盖、但对真实落地至关重要的配套设施缺口。这些缺口反过来印证了 §7.2 中的判断——AI Native 不是口号，而是需要业务与技术深度融合的系统工程。如果缺少统一语言的对齐、资源配置的支撑和 ROI 衡量的验证，零散 AI 实践很难自发整合为清晰的演进路径。

缺口一：子工程划分与 Monorepo 策略

挽单系统并非单一工程，而是由前端 H5、后端服务、移动端小程序、配置平台等多个子工程组成。当前 AINMM 的评估维度主要面向单一仓库，对于"跨仓库的上下文如何统一""多端变更如何联动验证""子工程间的依赖如何被 AI 理解"等问题，还需要在 Harness Layer 1（上下文工程）中进一步扩展。

缺口二：电商场景全自动测试链路

挽单承载的是售后挽回业务，涉及复杂的电商状态机（订单状态、退款状态、权益等）。当前的 Gate 3-4（单元测试 / 集成测试）能够验证代码层面的正确性，但**业务场景层面的全自动验证**仍然依赖人工构造数据和人工判断结果。构建让 AI 能自动生成符合业务规则的测试数据、自动验证端到端业务流程——是 ML3→ML4 阶段必须攻克的关卡。

缺口三：线上数据回流与运维闭环

当前 AI 的上下文主要停留在"开发态"，对于"线上异常如何触发开发态修复""运维数据如何反哺 AI 的上下文理解"还缺乏机制。公司管理层提出的"每天一个版本"不仅要求开发快，更要求**发布-监控-修复的闭环快**。这需要在 Harness Layer 5（自进化）中建立线上数据回流通道。

这些缺口不会让我否定 AINMM 的价值，反而让我认识到：**通用框架提供的是"地图"，具体团队需要结合业务特点在地图上标注自己的"险滩"**。AINMM 的当前版本是一个起点，而非终点。

未来演进与计划：向自进化系统迈进

上节识别的三个缺口——子工程上下文贯通、电商场景全自动验证、线上数据回流闭环——正是挽单向 ML3-ML5 演进需要重点攻克的方向。基于此，挽单的终极愿景，不是一个"AI 辅助开发"的系统，而是一个**自进化的 AI Native 系统**。基于当前的 ML2 基础和未来 ML3-ML5 的演进路径，我设想的完整闭环如下：

```
需求输入 → 自动 Spec 生成 → 自动开发 → 自动部署
    → 自动验证 → 自动回归 → 自动上线 → 自动运维
    → 线上问题自动收集 → 自动优化 → 需求再输入
```

在这个闭环中，**人只在必要时介入**：

- **需求收敛阶段**：人负责判断"做什么"和"为什么做"，AI 负责将需求转化为结构化 Spec
- **开发阶段**：AI 基于 Spec 自动生成代码、自动走查、自动修复门禁问题
- **验证阶段**：AI 自动构造测试数据、自动执行全链路回归、自动生成测试报告
- **上线阶段**：AI 自动灰度发布、自动监控核心指标、自动回滚异常变更
- **运维阶段**：AI 自动收集线上异常、自动归因、自动产生修复提案



这个愿景不是空想。ML1-ML2 已经走通了"自动开发 + 自动验证"的小闭环，ML3-ML4 将走通"自动部署 + 自动上线"的中闭环，ML5 将实现"自动运维 + 自动优化"的大闭环。每一步的演进都有 AINMM 的等级定义作为路标，都有审计脚本（audit.sh）的量化评分作为验证。接下来会在AI端到端自动化验证、尝试复用SRE Agent运维自进化的数据打通几个重点方向深入。

案例小结

挽单的案例对 AINMM 的价值，不是"证明了这个框架完美无缺"，而是**验证了"案例驱动框架、框架反哺案例"的双循环方法论是可行的**。

具体来说：

1. **定位能力得到验证**——五维度评分精确揭示了"D1/D2 领先、D3/D4 滞后"的不均衡状态，避免了"盲目堆 AI 工具"的陷阱
2. **路径指导能力得到验证**——评估结果直接指出"PA4 协作契约是最大短板"，让有限的研发资源投向了最关键的方向
3. **度量能力得到验证**——改进前后总分从 30 提升到 48，等级从 ML1 提升到 ML2，进步是可量化、可验证的
4. **框架的通用性得到验证**——挽单踩过的坑（Skill 缺少验证标准、门禁缺少业务场景覆盖、上下文缺少跨仓库联动）被抽象为 AINMM 的改进方向，让下一个项目可以避免同样的弯路

最后，回到前言中的那句话：**没有一种范式是银弹**。AINMM 和挽单的实践都是"进行时"，而非"完成时"。框架会持续进化，挽单会持续演进，我也将持续在真实的工程土壤中打磨这套方法。欢迎有兴趣的一起探索。

10 附录

附录 A：术语表

术语	全称	定义
AINMM	AI Native Maturity Model	AI Native 能力成熟度模型
AINA	AI Native Assessment	AI Native 评估方法
ML	Maturity Level	成熟度等级
PA	Process Area	过程域
SG	Specific Goal	特定目标
SP	Specific Practice	特定实践
GG	Generic Goal	通用目标
Harness Gap	—	AI 代码产出与可靠交付之间的缺口
Rippable	—	可随模型能力提升安全剥离的 Harness 控制

11 参考资料

[1] Sonar. State of Code Developer Survey Report: The current reality of AI coding. SonarSource Blog, 2025.

https://www.sonarsource.com/blog/state-of-code-developer-survey-report-the-current-reality-of-ai-coding

> 基于对 1,100 余名企业开发者的调查，报告指出 42% 的新增代码由 AI 辅助生成，并揭示了 AI 代码的验证缺口（Verification Gap）。

[2] GitClear. AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Duplicate Code. GitClear Research, 2025.

https://www.gitclear.com/ai_assistant_code_quality_2025_research

> 基于 1.53 亿行代码的纵向分析，研究发现 AI 辅助代码的搅动率（churn rate）近乎翻倍，重复代码增长近四倍，开发者花在 Review 上的时间首次超过编写代码的时间。（注：该链接为 GitClear 官方研究报告页面，部分网络环境可能受限。）

[3] Stack Overflow. Mind the gap: Closing the AI trust gap for developers. Stack Overflow Blog, 2026-02-18.

https://stackoverflow.blog/2026/02/18/closing-the-developer-ai-trust-gap/

> 基于 Stack Overflow 2025 开发者调查（数万名受访者），84% 的开发人员已在使用或计划使用 AI 编码工具，但仅有 29% 表示信任 AI 产出，信任度较上年下滑 11 个百分点。

[4] OpenAI. Harness engineering: leveraging Codex in an agent-first world. OpenAI Blog, 2026.

https://openai.com/index/harness-engineering/

> OpenAI 于 2026 年 2 月正式提出 Harness Engineering 理念，将"AI 代码产出能力快速增长但端到端交付效率未同步提升"的结构性缺口定义为 Harness Gap。

[5] CMMI Institute. CMMI for Development, Version 2.0. CMMI Institute, 2018.

https://cmmiinstitute.com/

> CMMI（能力成熟度模型集成）是软件工程领域广泛使用的过程改进框架，AINMM 继承其五级递进结构和关键过程域思想，并将其特化到 AI Native 研发范式。

> “**可测量意味着可管理，可管理意味着可优化**。”AINMM 的目标不是给个人/组织贴标签，而是试图给个人/组织一张地图——知道自己在哪、要去哪、以及怎么走到那里。



本文作者木直，来自淘天集团-供给技术团队。本团队是淘天技术中支撑商家、运营和行业商业模式的技术团队，承载了「技术驱动商业革新」的使命。团队支撑的业务能力覆盖电商全链路，从产业分析到智能运营决策，从商家经营自动化到生态创新，从行业消费者体验提升到产业链模式创新，覆盖了服饰、快消、消电、企业服务等各个行业，构建着智能驱动的商业革新引擎。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 2392

留言

写留言

还没有留言，来写首评